FILE: C:\HCL_TRAINING\server\src\controllers\jobController.js

LINES: 317 | SIZE: 7.6 KB | LANGUAGE: JS

MODULE: Job Opening Lifecycle (Create, Update, Duplicate, Readiness)

```
1 | const Job = require('../models/Job');
2 | const Application = require('../models/Application');
3 | const { analyzeJobReadiness, generateJobEnhancements } = require('../services/jobIntelli
  > gence/jobAnalyzer');
4 | const { logActivity } = require('../services/auditLogger');
5 |
6 | // @desc    Get all jobs for organization
7 | // @route   GET /api/jobs
8 | const getJobs = async (req, res, next) => {
9 |   try {
10 |     const { status, search, department } = req.query;
11 |     const filter = { organization: req.organizationId };
12 |
13 |     if (status && status !== 'all') {
14 |       filter.status = status;
15 |     }
16 |     if (department && department !== 'all') {
17 |       filter.department = department;
18 |     }
19 |     if (search) {
20 |       filter.$or = [
21 |         { title: { $regex: search, $options: 'i' } },
22 |         { department: { $regex: search, $options: 'i' } },
23 |         { location: { $regex: search, $options: 'i' } },
24 |       ];
25 |     }
26 |
27 |     const jobs = await Job.find(filter)
28 |       .sort({ createdAt: -1 })
29 |       .populate('createdBy', 'name email');
30 |
31 |     // Attach candidate counts per job
32 |     const jobsWithCounts = await Promise.all(
33 |       jobs.map(async (job) => {
34 |         const totalCandidates = await Application.countDocuments({ jobId: job._id });
35 |         const shortlistedCandidates = await Application.countDocuments({
36 |           jobId: job._id,
37 |           stage: { $in: ['Shortlisted', 'Interview', 'Offer', 'Hired'] },
38 |         });
39 |         return {
40 |           ...job.toObject(),
41 |           totalCandidates,
42 |           shortlistedCandidates,
43 |         };
44 |       })
45 |     );
46 |
47 |     res.status(200).json({
48 |       success: true,
49 |       count: jobsWithCounts.length,
50 |       data: jobsWithCounts,
51 |     });
52 |   } catch (error) {
53 |     next(error);
54 |   }
55 | };
56 |
57 | // @desc    Get single job by ID
58 | // @route   GET /api/jobs/:id
59 | const getJobById = async (req, res, next) => {
60 |   try {
61 |     const job = await Job.findOne({
62 |       _id: req.params.id,
63 |       organization: req.organizationId,
64 |     }).populate('createdBy', 'name email');
65 |
```

```
66 |       if (!job) {
67 |         return res.status(404).json({ success: false, message: 'Job not found' });
68 |       }
69 |
70 |       const totalCandidates = await Application.countDocuments({ jobId: job._id });
71 |       const stagesBreakdown = await Application.aggregate([
72 |         { $match: { jobId: job._id } },
73 |         { $group: { _id: '$stage', count: { $sum: 1 } } },
74 |       ]);
75 |
76 |       res.status(200).json({
77 |         success: true,
78 |         data: {
79 |           ...job.toObject(),
80 |           totalCandidates,
81 |           stagesBreakdown,
82 |         },
83 |       });
84 |     } catch (error) {
85 |       next(error);
86 |     }
87 |   };
88 |
89 |   // @desc    Create new job
90 |   // @route   POST /api/jobs
91 |   const createJob = async (req, res, next) => {
92 |     try {
93 |       const jobData = {
94 |         ...req.body,
95 |         organization: req.organizationId,
96 |         createdBy: req.user._id,
97 |       };
98 |
99 |       // Run Readiness Analysis
100 |      const readiness = analyzeJobReadiness(jobData);
101 |      jobData.readinessScore = readiness.clarityScore;
102 |      jobData.readinessAnalysis = readiness;
103 |
104 |      const job = await Job.create(jobData);
105 |
106 |      await logActivity({
107 |        organizationId: req.organizationId,
108 |        userId: req.user._id,
109 |        action: 'JOB_CREATED',
110 |        entityType: 'Job',
111 |        entityId: job._id,
112 |        details: { title: job.title, department: job.department },
113 |        req,
114 |      });
115 |
116 |      res.status(201).json({
117 |        success: true,
118 |        data: job,
119 |      });
120 |    } catch (error) {
121 |      next(error);
122 |    }
123 |  };
124 |
125 |  // @desc    Update existing job
126 |  // @route   PUT /api/jobs/:id
127 |  const updateJob = async (req, res, next) => {
128 |    try {
129 |      let job = await Job.findOne({
130 |        _id: req.params.id,
131 |        organization: req.organizationId,
132 |      });
133 |
134 |      if (!job) {
135 |        return res.status(404).json({ success: false, message: 'Job not found' });
136 |      }
137 |
```

```
138 |      // Recalculate readiness
139 |      const updatedData = { ...req.body };
140 |      const readiness = analyzeJobReadiness({ ...job.toObject(), ...updatedData });
141 |      updatedData.readinessScore = readiness.clarityScore;
142 |      updatedData.readinessAnalysis = readiness;
143 |
144 |      job = await Job.findByIdAndUpdate(req.params.id, updatedData, {
145 |        new: true,
146 |        runValidators: true,
147 |      });
148 |
149 |      await logActivity({
150 |        organizationId: req.organizationId,
151 |        userId: req.user._id,
152 |        action: 'JOB_UPDATED',
153 |        entityType: 'Job',
154 |        entityId: job._id,
155 |        details: { title: job.title },
156 |        req,
157 |      });
158 |
159 |      res.status(200).json({
160 |        success: true,
161 |        data: job,
162 |      });
163 |    } catch (error) {
164 |      next(error);
165 |    }
166 |  };
167 |
168 |  // @desc    Duplicate job
169 |  // @route   POST /api/jobs/:id/duplicate
170 |  const duplicateJob = async (req, res, next) => {
171 |    try {
172 |      const original = await Job.findOne({
173 |        _id: req.params.id,
174 |        organization: req.organizationId,
175 |      });
176 |
177 |      if (!original) {
178 |        return res.status(404).json({ success: false, message: 'Original job not found' })
    >  ;
179 |      }
180 |
181 |      const duplicateData = original.toObject();
182 |      delete duplicateData._id;
183 |      delete duplicateData.createdAt;
184 |      delete duplicateData.updatedAt;
185 |
186 |      duplicateData.title = `${original.title} (Copy)`;
187 |      duplicateData.status = 'draft';
188 |      duplicateData.createdBy = req.user._id;
189 |
190 |      const newJob = await Job.create(duplicateData);
191 |
192 |      await logActivity({
193 |        organizationId: req.organizationId,
194 |        userId: req.user._id,
195 |        action: 'JOB_DUPLICATED',
196 |        entityType: 'Job',
197 |        entityId: newJob._id,
198 |        details: { originalId: original._id, newTitle: newJob.title },
199 |        req,
200 |      });
201 |
202 |      res.status(201).json({
203 |        success: true,
204 |        data: newJob,
205 |      });
206 |    } catch (error) {
207 |      next(error);
208 |    }
```

```
209 |  };
210 |
211 |  // @desc    Archive or toggle status of job
212 |  // @route   PATCH /api/jobs/:id/status
213 |  const updateJobStatus = async (req, res, next) => {
214 |    try {
215 |      const { status } = req.body;
216 |      const job = await Job.findOneAndUpdate(
217 |        { _id: req.params.id, organization: req.organizationId },
218 |        { status },
219 |        { new: true }
220 |      );
221 |
222 |      if (!job) {
223 |        return res.status(404).json({ success: false, message: 'Job not found' });
224 |      }
225 |
226 |      await logActivity({
227 |        organizationId: req.organizationId,
228 |        userId: req.user._id,
229 |        action: `JOB_STATUS_${status.toUpperCase()}`,
230 |        entityType: 'Job',
231 |        entityId: job._id,
232 |        details: { status },
233 |        req,
234 |      });
235 |
236 |      res.status(200).json({
237 |        success: true,
238 |        data: job,
239 |      });
240 |    } catch (error) {
241 |      next(error);
242 |    }
243 |  };
244 |
245 |  // @desc    Delete job
246 |  // @route   DELETE /api/jobs/:id
247 |  const deleteJob = async (req, res, next) => {
248 |    try {
249 |      const job = await Job.findOneAndDelete({
250 |        _id: req.params.id,
251 |        organization: req.organizationId,
252 |      });
253 |
254 |      if (!job) {
255 |        return res.status(404).json({ success: false, message: 'Job not found' });
256 |      }
257 |
258 |      await logActivity({
259 |        organizationId: req.organizationId,
260 |        userId: req.user._id,
261 |        action: 'JOB_DELETED',
262 |        entityType: 'Job',
263 |        entityId: req.params.id,
264 |        details: { title: job.title },
265 |        req,
266 |      });
267 |
268 |      res.status(200).json({
269 |        success: true,
270 |        message: 'Job removed successfully',
271 |      });
272 |    } catch (error) {
273 |      next(error);
274 |    }
275 |  };
276 |
277 |  // @desc    Analyze raw job text or draft parameters
278 |  // @route   POST /api/jobs/analyze
279 |  const analyzeJob = async (req, res, next) => {
280 |    try {
```

```
281 |      const readiness = analyzeJobReadiness(req.body);
282 |      res.status(200).json({
283 |        success: true,
284 |        data: readiness,
285 |      });
286 |    } catch (error) {
287 |      next(error);
288 |    }
289 |  };
290 |
291 |  // @desc    Generate AI enhancements and skills for a job title
292 |  // @route   POST /api/jobs/enhance
293 |  const enhanceJobWithAI = async (req, res, next) => {
294 |    try {
295 |      const { title } = req.body;
296 |      const suggestions = generateJobEnhancements(title);
297 |      res.status(200).json({
298 |        success: true,
299 |        data: suggestions,
300 |      });
301 |    } catch (error) {
302 |      next(error);
303 |    }
304 |  };
305 |
306 |  module.exports = {
307 |    getJobs,
308 |    getJobById,
309 |    createJob,
310 |    updateJob,
311 |    duplicateJob,
312 |    updateJobStatus,
313 |    deleteJob,
314 |    analyzeJob,
315 |    enhanceJobWithAI,
316 |  };
317 |
```